# 1 Role of JavaScript & Embedding

- HTML = structure, CSS = looks, **JavaScript = behavior**.
- JS can react to clicks, validate forms, change content, talk to servers.

### Three ways to add a script

- **Inline:** inside an HTML element (use sparingly).
- **Internal:** inside a <script> tag in the HTML file.
- **External:** separate .js file linked with src (best for bigger code).

```
<script>
  const btn = document.querySelector('#btn');
  btn.textContent = 'Ready!';
</script>

<!-- External file -->
<script src="app.js"></script>
```

**Good practice:** keep larger JavaScript in external .js files.

## Script Loading: defer vs async

### defer

- Downloads while HTML is being parsed.
- Runs AFTER parsing is complete.
- Deferred scripts keep their **order**.
- Good when the script needs the page DOM.

### async

- Downloads while HTML is being parsed.
- Runs AS SOON AS download finishes.
- Order is **not guaranteed**.
- Good for independent scripts, e.g. analytics.

```
<script src="app.js" defer></script>
<script src="analytics.js" async></script>
```

## ② var, let and const

- **let** - value will change.
- **const** - should not be reassigned.
- **var** - old syntax, function-scoped, can be redeclared.
- Modern code: prefer const, then let when reassigning.
- const does NOT make an object immutable - only the reference can't be reassigned.

```
let score = 10;
score = 20;

const name = 'Ali';
// name = 'Sara'; // Error

var oldStyle = true;
```

**Rule of thumb:** start with const. Change to let only when you need reassignment.

## Primitive vs Reference Types

### Primitive (copied as VALUES)

- string, number, bigint, boolean, undefined, null, symbol
- Changing one copy does not change the other.

### Reference (variable holds a REFERENCE)

- Objects, arrays and functions.

- Two variables can point to the same object - a change shows through both.

```
let a = 10;
let b = a;
b = 20;
console.log(a); // 10

const user1 = { name: 'Ali' };
const user2 = user1;
user2.name = 'Sara';
console.log(user1.name); // Sara
```

## 3 Operators, Coercion, == vs ===

- **Arithmetic:** + - * / % **
- **Comparison:** > < >= <= == === != !==
- **Logical:** && (and), || (or), ! (not)
- **Assignment:** = += -= *= /=

### Type coercion

JS converts a value to another type in some operations. == allows conversion; === checks value AND type.

```
console.log(5 + '2');  // '52'
console.log(5 - '2');  // 3

console.log(5 == '5'); // true
console.log(5 === '5'); // false

// Prefer === in most code
if (age === 18) {
  console.log('Adult');
```

```
}
```

**Tip:** use === and !== by default - more predictable.

(+ with a string joins text -> '52', but - forces a number -> 3)

## 4 Control Flow

- **if** runs code when condition is true; **else** the other case; **else if** another check.
- **for** - known number of repeats. **while** - repeat while condition true.
- **switch** - choose among several cases. **break** exits a loop or switch.

```
const mark = 72;
if (mark >= 50) {
  console.log('Pass');
} else {
  console.log('Fail');
}

for (let i = 1; i <= 3; i++) {
  console.log(i);
}
```

```
const day = 2;
switch (day) {
  case 1: console.log('Mon'); break;
  case 2: console.log('Tue'); break;
  default: console.log('Other');
}
```

**Watch out:** forget break in a switch and it keeps running into the next case!

## 5 Scope: Global, Function, Block

- Scope = where a variable can be accessed.
- **Global:** whole script (avoid unnecessary globals).
- **Function:** var inside a function stays in that function.
- **Block:** let/const inside { } live only in that block.
- Block scope prevents accidental changes.

```
const globalName = 'Ali';

function test() {
  var x = 10;      // function scope
  if (x > 5) {
    let y = 20;    // block scope
    const z = 30;  // block scope
  }
  console.log(x);  // OK
  // console.log(y); // Error
}
```

## 6 Hoisting & the TDZ

### Hoisting

- JS processes declarations BEFORE running code.
- Function declarations can be called before they're written.
- var is hoisted and starts as **undefined**.
- Hoisted does not mean safe to use early!

### Temporal Dead Zone

- let & const are hoisted but NOT initialized.
- Time before the declaration line = the TDZ.
- Using them there -> **ReferenceError**.
- Declare variables before using them - it's clearer.

```
sayHello();
function sayHello() {
  console.log('Hello');
}

console.log(a); // undefined
var a = 10;

// console.log(b); // ReferenceError
let b = 20;
```

## 7 Functions: 3 Forms

- **Declaration:** named function written with function.
- **Expression:** function stored in a variable.
- **Arrow:** shorter syntax, great for callbacks.
- Functions take parameters and return values.
- Arrow functions have NO own this.

```
// Declaration
function add(a, b) {
  return a + b;
}

// Expression
const multiply = function(a, b) {
  return a * b;
};

// Arrow
const square = n => n * n;
```

```
console.log(add(2, 3)); // 5
```

## 8 Default & Rest Parameters

- **Default:** value used when argument is missing - written in the parameter list.
- **Rest:** ... before the name; collects remaining args into an array; must be LAST.

```
function greet(name = 'Guest') {
  return `Hello, ${name}`;
}
console.log(greet());
// Hello, Guest

function total(...numbers) {
  return numbers.reduce((sum, n) => sum + n, 0);
}
console.log(total(2, 3, 4)); // 9
```

## 9 Spread & Destructuring

- **Spread (...)** expands an array/object into single values/properties.
- **Destructuring** pulls values out into variables.
- Array destructuring -> by POSITION. Object destructuring -> by property NAME.

**Don't mix up:** rest COLLECTS values into an array; spread EXPANDS them out.

```
const nums = [1, 2, 3];
const more = [...nums, 4, 5];

const user = { name: 'Ali', age: 20 };
const { name, age } = user;

const [first, second] = nums;
console.log(more); // [1,2,3,4,5]
```

## ⑩ this, call, apply, bind

- this depends on HOW a function is called.
- obj.method() -> this is usually obj.
- **call()** - run now, args separated by commas.
- **apply()** - run now, args in an ARRAY.
- **bind()** - returns a NEW function with this fixed (run later).

```
const person = { name: 'Ali' };

function greet(city) {
  console.log(this.name + ' from ' + city);
}

greet.call(person, 'Abbottabad');
greet.apply(person, ['Lahore']);

const fixed = greet.bind(person);
fixed('Islamabad');
```

**Remember:** call/apply run now; bind returns a new function to run later.

# 11 Closures

- Inner function remembers variables from its outer scope.
- **Lexical scope:** function uses the scope where it was CREATED.
- Uses: private state, counters, callbacks, function factories.
- Pitfall: can keep data alive longer than expected.
- Keep closures simple; don't retain unneeded data.

```
function createCounter() {
  let count = 0;

  return function () {
    count++;
    return count;
  };
}

const counter = createCounter();
console.log(counter()); // 1
console.log(counter()); // 2
```

**Why it works:** the returned function still remembers count after createCounter() has finished.

# 12 Objects and Arrays

## Objects

- Key-value pairs of related data.
- Access with dot (obj.name) or bracket (obj['name']).
- Methods = functions stored as properties.
- Objects are reference values.

## Arrays

- Ordered collections, index starts at 0.

- *Methods: push, pop, map, filter, find.*
- *Arrays are also objects.*

```
const student = {
  name: 'Ayesha',
  marks: 85,
  passed: true
};

const marks = [70, 85, 90];
const high = marks.filter(m => m >= 80);

console.log(student.name);
console.log(high); // [85, 90]
```

## 13 Template Literals

- *Use backticks: `...`*
- *Insert variables/expressions with ${...}.*
- *Can span multiple lines.*
- *Great for messages, HTML strings, formatted output.*

```
const name = 'Ali';
const marks = 85;

const message = `Student: ${name}
Marks: ${marks}
Status: ${marks >= 50 ? 'Pass' : 'Fail'}`;

console.log(message);
```

**Tip:** *use template literals instead of lots of + when building long strings.*

## 14 ES6 Modules

- Split a big program into smaller files.
- **export** makes code available; **import** brings it in.
- Named exports -> names must match.
- **default export** = one main value per module.
- Helps organize and reuse code.

```
// math.js
export function add(a, b) {
  return a + b;
}
export const tax = 0.15;

// app.js
import { add, tax } from './math.js';

console.log(add(2, 3));
console.log(tax);
```

## 15 Putting It All Together

Mini project uses: const/let + object + array method + function with default param + template literal.

```
const student = {
  name: 'Sara',
  marks: [72, 85, 91]
};

function average(values = []) {
  return values.reduce((a, b) => a + b, 0) / values.length;
```

```
}

const avg = average(student.marks);
const result = avg >= 50 ? 'Pass' : 'Fail';

console.log(`${student.name}: ${avg.toFixed(1)} - ${result}`);
// Sara: 82.7 - Pass
```

# Quick Revision Sheet

| | |
|---|---|
| **let / const / var** | const -> let -> (avoid) var |
| **Primitive vs reference** | value copy vs shared object |
| **== vs ===** | === checks value + type |
| **var in TDZ?** | undefined (no error) |
| **let/const in TDZ?** | ReferenceError |
| **Function declaration early call** | works (hoisted) |
| **Arrow function** | no own this |
| **rest vs spread** | collect vs expand |
| **call / apply / bind** | now+args / now+array / new fn later |
| **defer vs async** | ordered after parse / runs asap, no order |
| **Closure** | inner fn remembers outer vars |
| **Object / array copy** | both are references! |

## Common mistakes to avoid

- Using var out of habit -> use const/let.
- Assuming const objects can't change inside.
- Using == and getting surprise conversions.
- Using let/const before the declaration line (TDZ).
- Forgetting break in switch.
- Putting rest parameter anywhere but last.
- Expecting async scripts to run in order.

**Study tip:** practice each snippet in the browser console - writing code beats re-reading!